Lab Assignment 8

📝 Aim

To design and implement a database system for hotel management applications that ensures efficient storage, retrieval, and manipulation of data related to customers, reservations, rooms, and services, thereby supporting smooth operational workflows and decision-making.

🎯 Objectives

- To understand the principles of database design and normalization.
- To create relational schemas for hotel management including entities such as customers, rooms, bookings, staff, and services.
- To implement SQL queries for data insertion, retrieval, and updates.
- To apply constraints (Primary Key, Foreign Key, Not Null, Unique) for maintaining data integrity.
- To design stored procedures, triggers, and views for automating hotel operations.
- To evaluate the performance and scalability of the database system in real-world scenarios.

⚙️ Requirements

- **Software/Tools:**
    - Relational Database Management System (RDBMS) such as MySQL, PostgreSQL, or Oracle DB
    - SQL client or GUI tool (e.g., MySQL Workbench, pgAdmin)
    - Application development environment (optional, e.g., Java/Python with JDBC/ODBC for integration)
- **System Setup:**
    - Computer with minimum 4 GB RAM and stable internet connection
    - Installed RDBMS with active database connection
- **Knowledge Prerequisites:**
    - Basic SQL syntax and relational database concepts
    - Understanding of data modeling (entities, attributes, relationships)
    - Familiarity with normalization techniques (1NF, 2NF, 3NF)

📖 Theory

Database systems are the backbone of modern applications, enabling structured storage and efficient retrieval of data. In hotel management, a database system integrates multiple modules such as room allocation, customer records, billing, and staff management.

- **Database Design:** Involves identifying entities (e.g., Customer, Room, Booking, Staff) and relationships (e.g., Customer–Booking, Room–Booking). Normalization ensures minimal redundancy and consistency.
- **Constraints:**
    - **Primary Key:** Ensures unique identification of records (e.g., Customer_ID).
    - **Foreign Key:** Maintains referential integrity between tables (e.g., Booking references Customer_ID).
    - **Not Null & Unique:** Enforces mandatory fields and uniqueness (e.g., Room_Number).
- **SQL Implementation:** SQL provides commands for creating tables, inserting records, and retrieving information. Advanced features like triggers automate tasks (e.g., updating room availability after booking), while views provide simplified access to complex queries.
- **Application Integration:** Database systems can be integrated with front-end applications to provide user-friendly interfaces for hotel staff and customers. This enhances efficiency, reduces manual errors, and supports decision-making through reports and analytics.

Code:

```
-- Create Customer table
CREATE TABLE Customer (
    Customer_ID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(15) UNIQUE NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Address VARCHAR(255)
);


-- Create Room table
CREATE TABLE Room (
    Room_ID INT PRIMARY KEY AUTO_INCREMENT,
    Room_Number VARCHAR(10) UNIQUE NOT NULL,
    Room_Type VARCHAR(50) NOT NULL,
    Price DECIMAL(10,2) NOT NULL,
    Status VARCHAR(20) DEFAULT 'Available'
```

```
);

-- Create Booking table
CREATE TABLE Booking (
    Booking_ID INT PRIMARY KEY AUTO_INCREMENT,
    Customer_ID INT NOT NULL,
    Room_ID INT NOT NULL,
    CheckIn DATE NOT NULL,
    CheckOut DATE NOT NULL,
    Total_Amount DECIMAL(10,2),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
    FOREIGN KEY (Room_ID) REFERENCES Room(Room_ID)
);

-- Create Staff table
CREATE TABLE Staff (
    Staff_ID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    Role VARCHAR(50) NOT NULL,
    Phone VARCHAR(15) UNIQUE NOT NULL
);

-- Create Services table
CREATE TABLE Services (
    Service_ID INT PRIMARY KEY AUTO_INCREMENT,
    Service_Name VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) NOT NULL
);

-- Create ServiceUsage table (linking services to bookings)
```

```
CREATE TABLE ServiceUsage (
    Usage_ID INT PRIMARY KEY AUTO_INCREMENT,
    Booking_ID INT NOT NULL,
    Service_ID INT NOT NULL,
    Quantity INT DEFAULT 1,
    FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID),
    FOREIGN KEY (Service_ID) REFERENCES Services(Service_ID)
);
-- Insert Customers
INSERT INTO Customer (Name, Phone, Email, Address)
VALUES ('Rajesh Sharma', '9876543210', 'rajesh@example.com', 'Shirpur, MH'),
       ('Anita Desai', '9123456780', 'anita@example.com', 'Mumbai, MH');

-- Insert Rooms
INSERT INTO Room (Room_Number, Room_Type, Price, Status)
VALUES ('101', 'Deluxe', 2500.00, 'Available'),
       ('102', 'Suite', 5000.00, 'Available');

-- Insert Staff
INSERT INTO Staff (Name, Role, Phone)
VALUES ('Ramesh Kumar', 'Receptionist', '9988776655'),
       ('Priya Singh', 'Housekeeping', '8877665544');

-- Insert Services
INSERT INTO Services (Service_Name, Price)
VALUES ('Laundry', 200.00),
       ('Breakfast Buffet', 500.00);
```

```
-- 1. View all available rooms
SELECT Room_Number, Room_Type, Price
FROM Room
WHERE Status = 'Available';

-- 2. Make a booking
INSERT INTO Booking (Customer_ID, Room_ID, CheckIn, CheckOut, Total_Amount)
VALUES (1, 1, '2026-10-15', '2026-10-18', 7500.00);

-- 3. Update room status after booking
UPDATE Room
SET Status = 'Booked'
WHERE Room_ID = 1;

-- 4. Add service usage for a booking
INSERT INTO ServiceUsage (Booking_ID, Service_ID, Quantity)
VALUES (1, 2, 2); -- Breakfast Buffet x2

-- 5. Generate customer bill (room + services)
SELECT c.Name, b.Total_Amount AS Room_Charge,
    SUM(s.Price * su.Quantity) AS Service_Charge,
    (b.Total_Amount + SUM(s.Price * su.Quantity)) AS Total_Bill
FROM Booking b
JOIN Customer c ON b.Customer_ID = c.Customer_ID
LEFT JOIN ServiceUsage su ON b.Booking_ID = su.Booking_ID
LEFT JOIN Services s ON su.Service_ID = s.Service_ID
WHERE b.Booking_ID = 1
GROUP BY c.Name, b.Total_Amount;
```

Output:

Conclusion: